

FRACMATH: A vectorized MATLAB framework for continuum damage mechanics with crack band regularization

Jaykumar Mavani^a, Madura Pathirage^{a,*}

^a*Gerald May Department of Civil, Construction, and Environmental Engineering, University of New Mexico, Albuquerque, NM, USA*

Abstract

FRACMATH is a vectorized MATLAB reference implementation of local isotropic damage with direction-dependent crack-band regularization. Its contribution is an inspectable workflow linking mesh import, projected width, constitutive history, sparse assembly, and solution. Material-point tests check strain mapping, irreversible damage, unloading, and fracture-energy calibration. A controlled three-mesh bending study compares regularized and fixed softening laws, measures partial structural dissipation, and checks load-increment sensitivity. Single-CPU Abaqus jobs provide response and diagnostic timing comparisons. Three-dimensional examples illustrate damage localization. Source code, response histories, energy diagnostics, and plotting scripts accompany the manuscript; algorithmic and validation limitations are stated.

Keywords: continuum damage mechanics, crack-band regularization, finite element method, MATLAB, Abaqus UMAT, quasibrittle fracture

Required Metadata

Current code version

Nr.	Code metadata description	Value
C1	Current code version	SoftwareX study, commit 98ad87d
C2	Permanent GitHub link to code/repository used for this code version	Source and study data: immutable GitHub commit .
C3	Legal Code License	MIT License
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	MATLAB; Fortran UMAT; Abaqus/Standard; Python analysis scripts
C6	Compilation requirements, operating environments & dependencies	MATLAB R2024b; Abaqus/Standard 2024 with Intel Fortran; Python 3 with NumPy, SciPy, Matplotlib, Pillow, and psutil. Verified on Windows 11.
C7	If available Link to developer documentation/manual	Repository README, implementation walkthrough, and reproduction guides
C8	Support email for questions	mpathirage@unm.edu

Table 1: Code metadata.

Motivation and significance

FRACMATH is intended for researchers and students who need to inspect the numerical path from a strain field to a damage field and a global response. The constitutive law and crack-band principle

*Corresponding author

Email address: mpathirage@unm.edu (Madura Pathirage)

are established methods [1, 2, 3]. The useful software question is whether the implementation makes its assumptions, update sequence, regularization length, and numerical limitations visible enough to check and modify. The package therefore includes element-level routines, a 2D bending case, two 3D examples, plotting scripts, and an Abaqus UMAT for independent comparison. The controlled study examines how element-size compensation affects structural response and dissipation, and how increment size and sparse linear algebra limit the numerical workflow.

Related software and scope

General-purpose finite element systems such as Abaqus [4] and OOFEM [5] provide broader element and analysis capabilities. CALFEM [6] provides an accessible MATLAB finite element teaching environment. FRACMATH offers a narrower, code-readable example of direction-dependent local crack-band damage in 2D and 3D. It is designed for teaching, replication, and testing method changes, not as a claim of faster or more accurate performance than those systems.

Software description

A scalar damage variable $\omega \in [0, 1]$ scales the elastic stress: $\boldsymbol{\sigma} = (1 - \omega)\mathbb{C}_0 : \boldsymbol{\varepsilon}$. The history variable $\kappa_{n+1} = \max(\kappa_n, \tilde{\varepsilon}_{n+1})$ makes the update irreversible. The modified von Mises measure is

$$\tilde{\varepsilon} = \frac{k-1}{2k(1-2\nu)}I_1 + \frac{1}{2k}\sqrt{\left(\frac{k-1}{1-2\nu}\right)^2 I_1^2 + \frac{12k}{(1+\nu)^2}J_2}, \quad J_2 = \frac{(e_1 - e_2)^2 + (e_2 - e_3)^2 + (e_3 - e_1)^2}{6}, \quad (1)$$

where $k = f_c/f_t$. For the 2D plane-stress element, $e_3 = -\nu(e_1 + e_2)/(1 - \nu)$. Both the MATLAB and UMAT implementations use Eq. (1).

The damage law is

$$\omega(\kappa) = 1 - \frac{\kappa_0}{\kappa} \exp\left[-\frac{\kappa - \kappa_0}{\varepsilon_f - \kappa_0}\right], \quad \kappa_0 = f_t/E, \quad \varepsilon_f = \frac{\kappa_0}{2} + \frac{G_F}{hf_t}. \quad (2)$$

The Oliver projected width for a three-node triangle is

$$h(\mathbf{n}) = \frac{2}{\sum_{a=1}^3 |\nabla N_a \cdot \mathbf{n}|}, \quad (3)$$

with $\mathbf{n}$ along maximum principal strain. The 3D tetrahedral implementation uses the analogous projected width. These are local regularization formulas; the material-point energy check below does not by itself prove structural mesh objectivity.

Numerical workflow and code map

The 2D driver `solver_main_3pb.m` reads the mesh and support/loading sets, precomputes elastic element matrices and shape-function gradients, and advances prescribed displacement. At each increment it (1) assembles the secant stiffness from the previous damage field, (2) factors its free block with `decomposition`, (3) solves equilibrium for the frozen damage state, (4) obtains element strains with `pagemtimes`, updates h , κ , and ω in `damage_update`, and (5) reassembles the damaged stiffness for reaction forces and the next increment. The factorization helper first attempts Cholesky and falls back to LU if that attempt fails. It is reused only within a displacement increment. There is no outer equilibrium iteration after the damage update, so this is a sequential secant scheme rather than a fully converged Newton solve. The post-update free-DOF residual is exported for each increment.

The optional Abaqus path uses `run_3pb_abaqus_OLIVER_T3_FAST.py` to write a table of T3 shape-function gradients. The Fortran `cdm_umat_2d_OLIVER_T3_FAST.for` reads that table to compute Eq. (3); it does not use `CELENT` when the table is present. The UMAT supplies the degraded elastic secant matrix as `DDSDDE`, rather than a consistent derivative of the evolving damage law. Its approximate tangent requires reporting equilibrium iteration counts with timings. Table initialization currently requires one Abaqus analysis thread.

Material-point verification

A one-element uniaxial test checks $\tilde{\varepsilon} = \varepsilon$ in tension and $\tilde{\varepsilon} = |\varepsilon|/k$ in compression, monotone damage under unload/reload, and $h \int \sigma d\varepsilon \approx G_F$ for four prescribed projected widths. Numerical integration to a large terminal strain gives the following results for $G_F = 0.090$ N/mm. A standalone driver linked to the unchanged Fortran UMAT repeats the strain-mapping, projected-width, energy, and unload/reload assertions; its energy results agree with the MATLAB values at the precision shown. Sixteen additional UMAT checks combine these four triangle widths with loading directions of 15° , 45° , 75° , and 90° . Projected widths agree with Eq. (3), and the largest absolute relative energy error is 1.78×10^{-6} . These are local implementation checks, not evidence of structural orientation independence.

Table 2: Material-point softening-energy checks. Oliver relative error is against $G_F = 0.090$ N/mm. The fixed-law integral targets $G_F h/h_{\text{ref}}$ with $h_{\text{ref}} = 1.25$ mm. These are local checks.

h (mm)	Oliver integral (N/mm)	Relative error	Fixed-law integral (N/mm)
0.5	0.090000160	1.78×10^{-6}	0.036000064
1.0	0.090000160	1.77×10^{-6}	0.072000128
2.0	0.090000159	1.77×10^{-6}	0.144000255
4.0	0.090000159	1.76×10^{-6}	0.288000511

Illustrative examples

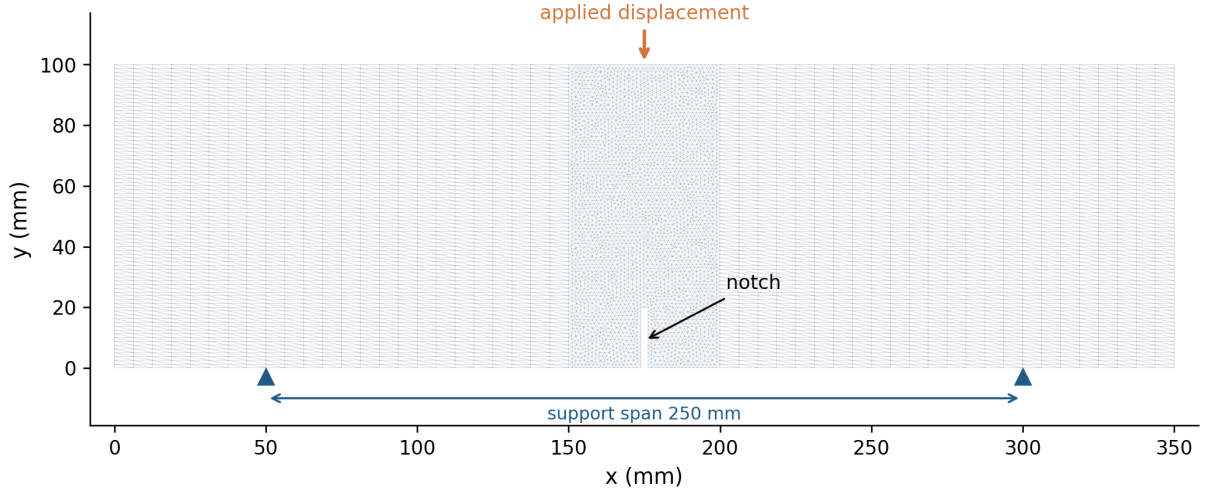
Benchmark 1: Notched 2D three-point bending

The Grégoire beam [7] has depth $D = 100$ mm, support span $S = 250$ mm, overall length 350 mm, thickness 50 mm, and a midspan notch of depth 20 mm and width 2.5 mm. Concrete parameters are $E = 37,000$ MPa, $\nu = 0.20$, $f_t = 3.5$ MPa, $f_c = 35$ MPa, and $G_F = 0.090$ N/mm. The 14,268 CPS3-element mesh has 7,319 nodes. The left support is pinned, the right is a roller, and a prescribed vertical displacement reaches -0.2 mm. CMOD is the mean horizontal displacement difference across the notch mouth. Figure 1 uses the mesh drawing: the 250 mm dimension denotes support span, while 350 mm is the overall length.

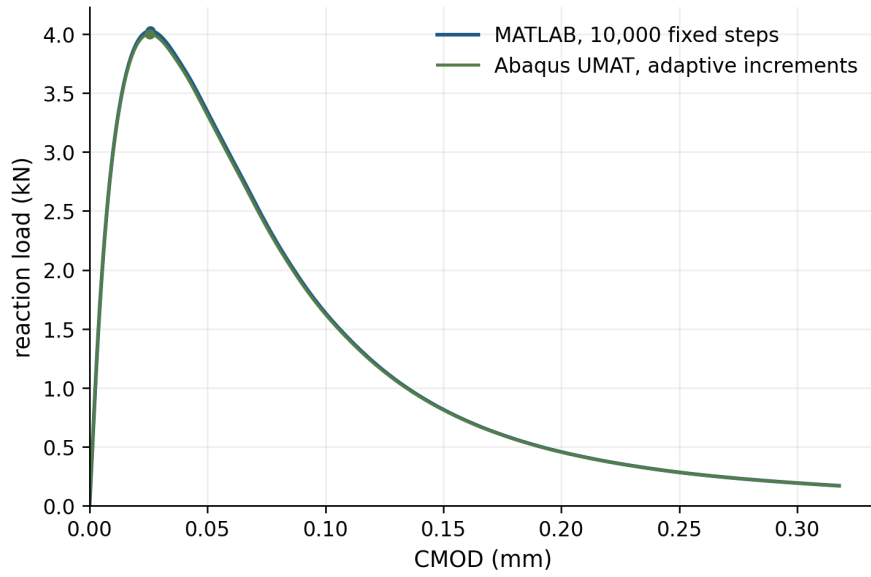
The comparison in Fig. 1 uses the 10,000-step MATLAB response and the Abaqus UMAT response. The separate increment-sensitivity check found that the 1,000-step MATLAB run peaks at 4.26 kN (CMOD 0.02835 mm, step 174), while the 10,000-step run peaks at 4.03 kN (CMOD 0.02579 mm, step 1612). This 5.9% change in peak load shows that increment size matters for the sequential scheme; these results are numerical examples, not an increment-converged validation against experiment. The normalized post-update free-DOF residual at the peak is 0.016 for 1,000 steps and 0.002 for 10,000 steps. A single-thread Abaqus/Standard 2024 UMAT run produced a peak of 4.00 kN at CMOD 0.02540 mm. The 10,000-step MATLAB peak differs by 0.68% in load and 1.51% in peak CMOD. The close response is a code-to-code check, not experimental validation.

The completed Abaqus .msg reports 1,136 accepted increments, 4,817 equilibrium iterations/solver passes, 4,799 matrix decompositions, one cutback, and 45 acceptances under the alternate force tolerance. Its summed per-pass sparse-solver elapsed time is 100.2 s out of 832 s reported wall clock. The remaining wall time includes element/UMAT operations, assembly, convergence and output, which the .msg does not separate. Thus no attribution of the remainder to the UMAT or speed ranking between codes is supported. The MATLAB run used 10,000 fixed increments, while Abaqus used automatic increments, so the two wall-clock totals are not a controlled benchmark. The repository now includes a parser that reproduces these counts and timings from the archived .msg and MATLAB timing file. New MATLAB runs export per-step component timings, iteration counts, and the post-damage equilibrium residual. The implementation walkthrough maps these measurements to the corresponding solver functions. The preserved 10,000-step run predates this per-step export; its aggregate timer and response data are retained without alteration.

Figure 2 compares damage fields at peak and at the first saved post-peak state above CMOD 0.30 mm. The Abaqus field output is taken from the nearest stored frame, while the MATLAB panels use the state arrays.



(a) Geometry, mesh, supports, and imposed displacement.



(b) MATLAB 10,000-step response and Abaqus adaptive-increment response.

Figure 1: 2D three-point bending case.

The 10,000-step run used MATLAB R2024b, one computational thread, and headless output on a Windows 11 workstation with an Intel Core i9-13900 CPU and 64 GB RAM. Timing scopes around stiffness assembly, free-block factorization, damage update, and triangular solves are recorded separately in Table 3 and Figure 3. Factorization is the largest measured MATLAB component at this problem size. The Abaqus sparse-solver time and the unallocated remainder are reported separately in Table 4. A cross-solver speed comparison requires matched increments, convergence quality, solver settings, and timing boundaries.

Table 3: Measured MATLAB 2D timing for the 10,000-step headless run. Other is total wall time minus the instrumented scopes.

Component	Time (s)	Share
Assembly	174.91	22.3%
Sparse factorization	519.54	66.1%
Damage update	17.95	2.3%
Free-block solve	71.52	9.1%
Other	1.78	0.2%
Total	785.70	100%

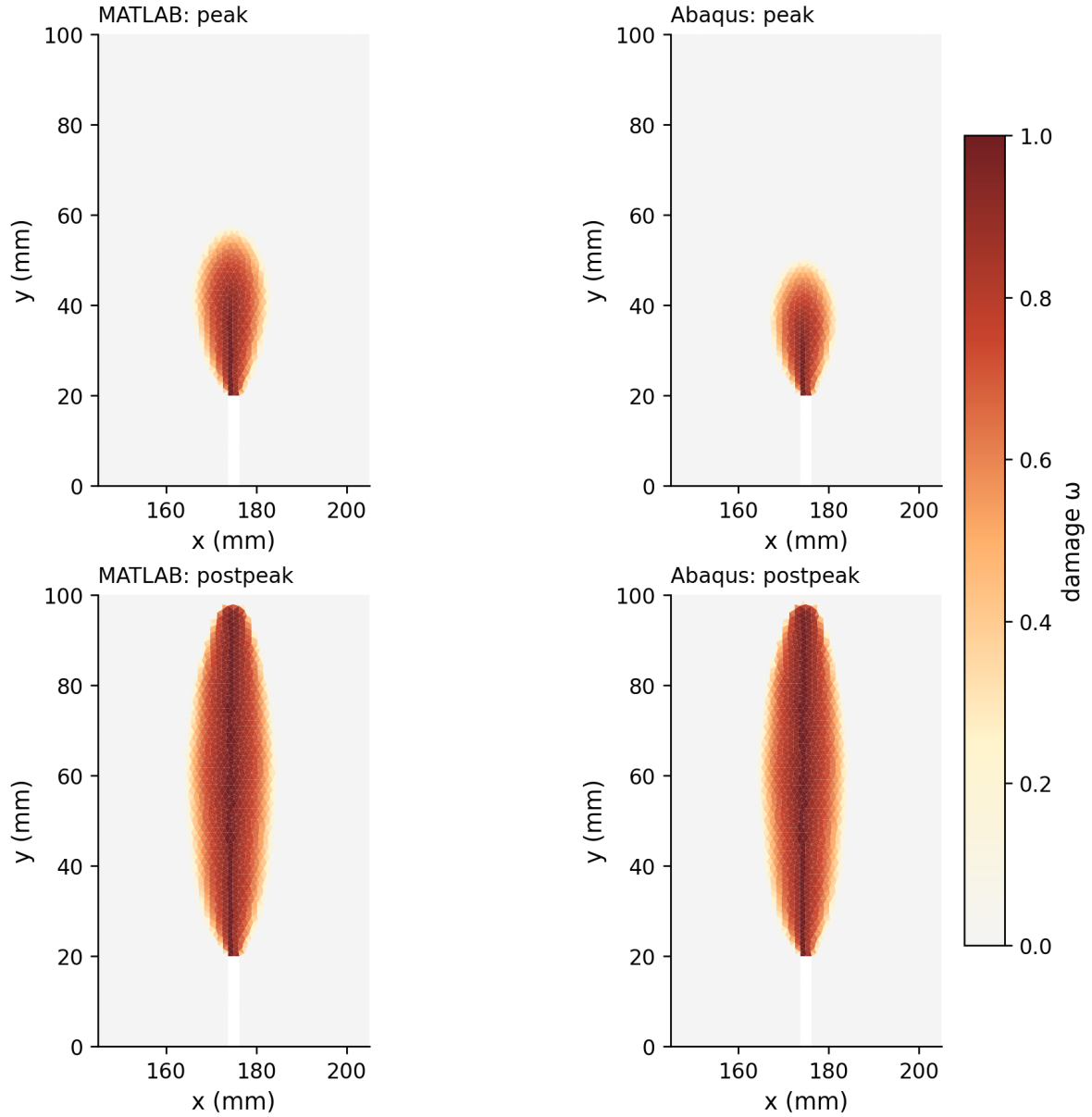


Figure 2: Damage field ω in the notch region on the undeformed CPS3 mesh. Left: MATLAB; right: Abaqus/UMAT. Top: near peak load; bottom: post-peak state. Abaqus fields are the closest stored frames to the indicated response points.

Table 4: Abaqus/Standard elapsed time extracted from the job’s `.msg`. The remaining time cannot be partitioned into UMAT, assembly, convergence, output, or overhead using this record.

Component	Time (s)	Share
Summed sparse-solver passes (4,817)	100.15	12.0%
Remaining wall time (unallocated)	731.85	88.0%
Total Abaqus wall clock	832.00	100%

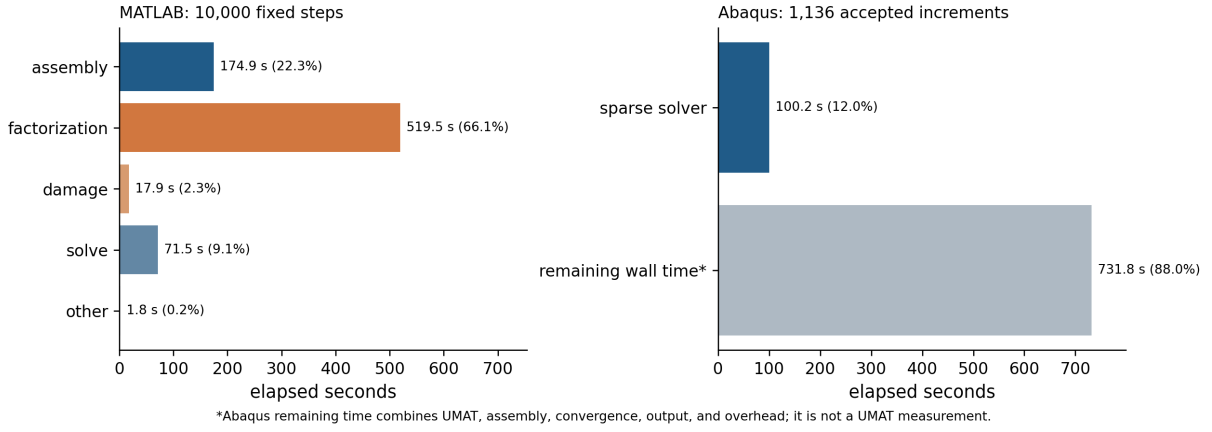


Figure 3: Measured timing components from the archived MATLAB run and Abaqus `.msg`. The remainder of the Abaqus wall time is unallocated; the plots do not establish a cross-code speed ranking.

Benchmark 2: 3D Nooru-Mohamed Mixed Mode Test

The Nooru-Mohamed specimen is a square double edge notched concrete panel, $200 \times 200 \times 50$ mm, with two horizontal notches 25 mm long cut from opposite edges at mid height [8]; the central ligament between the notch tips experiences a mixed mode stress state. We follow load path 4c: a vertical (mode I) displacement is prescribed on the top edge and a proportional horizontal (shear) displacement on the side edges, with a shear to tension ratio $\gamma = 0.6$ and the bottom edge held fixed, ramped together under displacement control over 900 increments to a top displacement of 0.5 mm. The material is concrete with $E = 29,000$ MPa, $\nu = 0.20$, $f_t = 3.0$ MPa, $G_F = 0.110$ N/mm, and $k = f_c/f_t = 10$ (so $f_c = 30$ MPa).

The 3D panels illustrate the damage workflow qualitatively; quantitative validation of this case is outside the scope of this study.

Benchmark 3: 3D Notched Beam Torsion

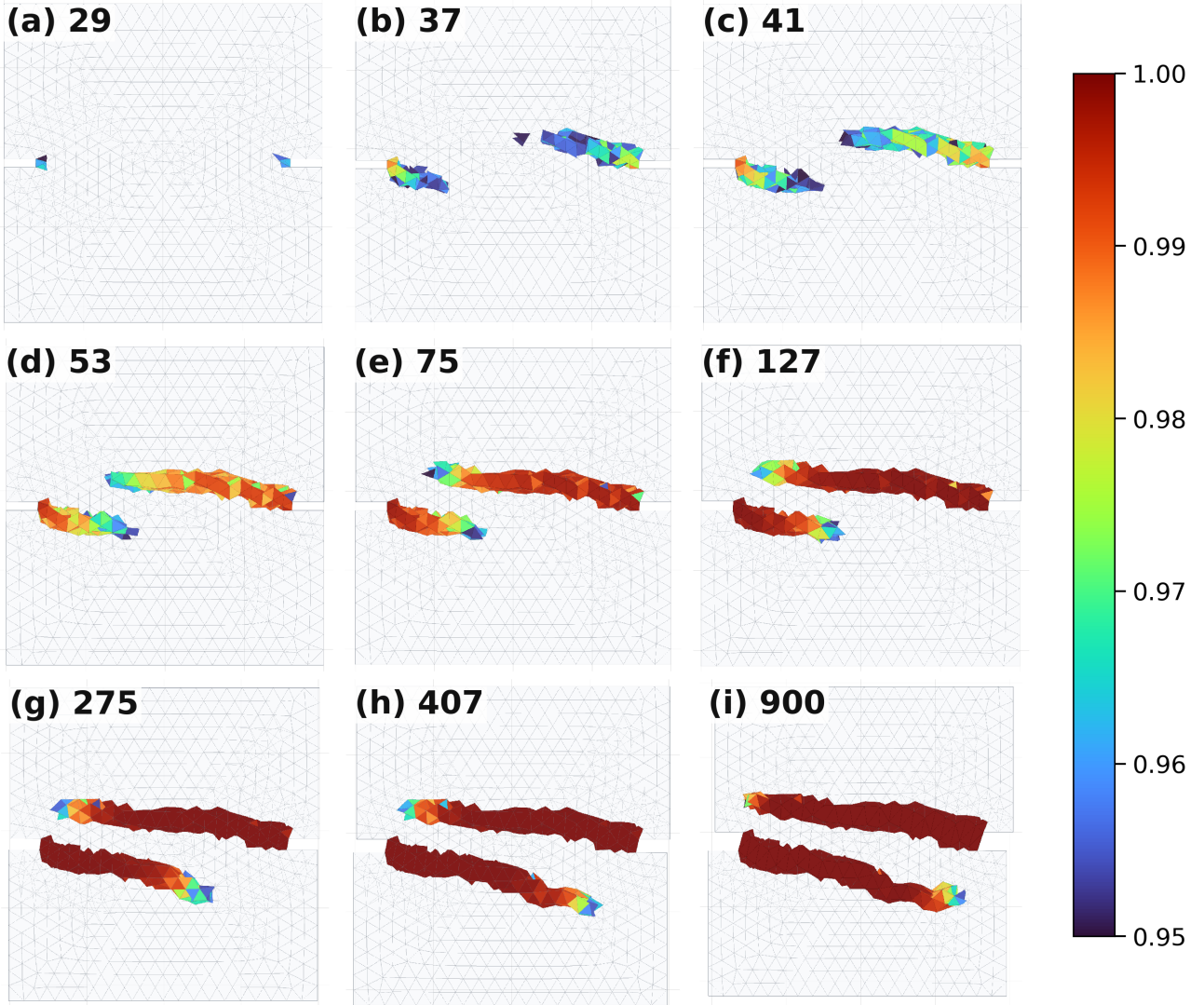
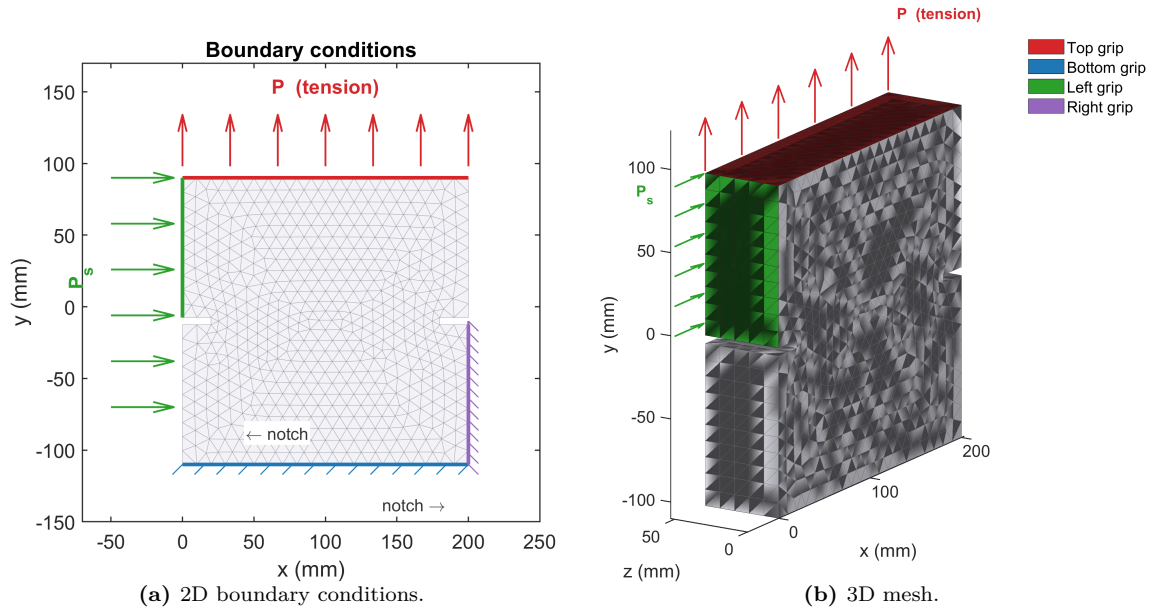
Brokenshire’s torsion test [9, 10] uses a $400 \times 250 \times 100$ mm plain concrete beam with a diagonal notch cut through the thickness (depth 25 mm, width 5 mm) across the bottom face at midspan. Three supports sit at the bottom corners and one end of the top edge, and a single point load acts on the remaining top corner (Figure 5a), so the beam is twisted and the plane of maximum principal tension turns in space around the notch front, lining up with no face of the mesh. The projected length is recomputed from the local principal direction at each integration point. This example illustrates a nonplanar damage path; its orientation and energy convergence require a mesh study.

In the model one end face is fully fixed while the opposite end face is given a prescribed rigid-body twist θ about the beam axis under rotation control, ramped to $\theta_{\text{end}} = 3.0 \times 10^{-3}$ rad over 140 increments; the reaction torque is recovered from the twisted end.

The material is plain concrete with $E = 35,000$ MPa, $\nu = 0.20$, $f_t = 3.0$ MPa, $G_F = 0.080$ N/mm, $\kappa_0 = 6.0 \times 10^{-5}$, and $k = f_c/f_t = 10$ (so $f_c = 30$ MPa), with the same exponential softening scaled element by element through the crack band length.

The TET4 mesh carries a refined layer around the notch front. The damage update is under-relaxed with a factor of 0.35 to keep the staggered fixed point iteration stable through the steep post-peak branch, and each linear solve reuses a single AMD ordering with a Cholesky factorization.

This case is a qualitative demonstration of direction-dependent crack-band regularization under nonplanar, torsion-driven crack growth: the predicted damage band nucleates at the notch front and spirals toward the loaded corner (Figure 5), qualitatively similar to the published fracture description [9]. A quantitative comparison against the measured load–crack-opening response is left for the planned nonlocal extension of the framework.



(c) Damage evolution from first localization at the notches to coalescence; panels (a)–(i), left to right and top to bottom, correspond to increments 29, 37, 41, 53, 75, 127, 275, 407, and 900.

Figure 4: Nooru-Mohamed benchmark: boundary conditions, 3D mesh, and damage evolution ω over the imposed loading history.

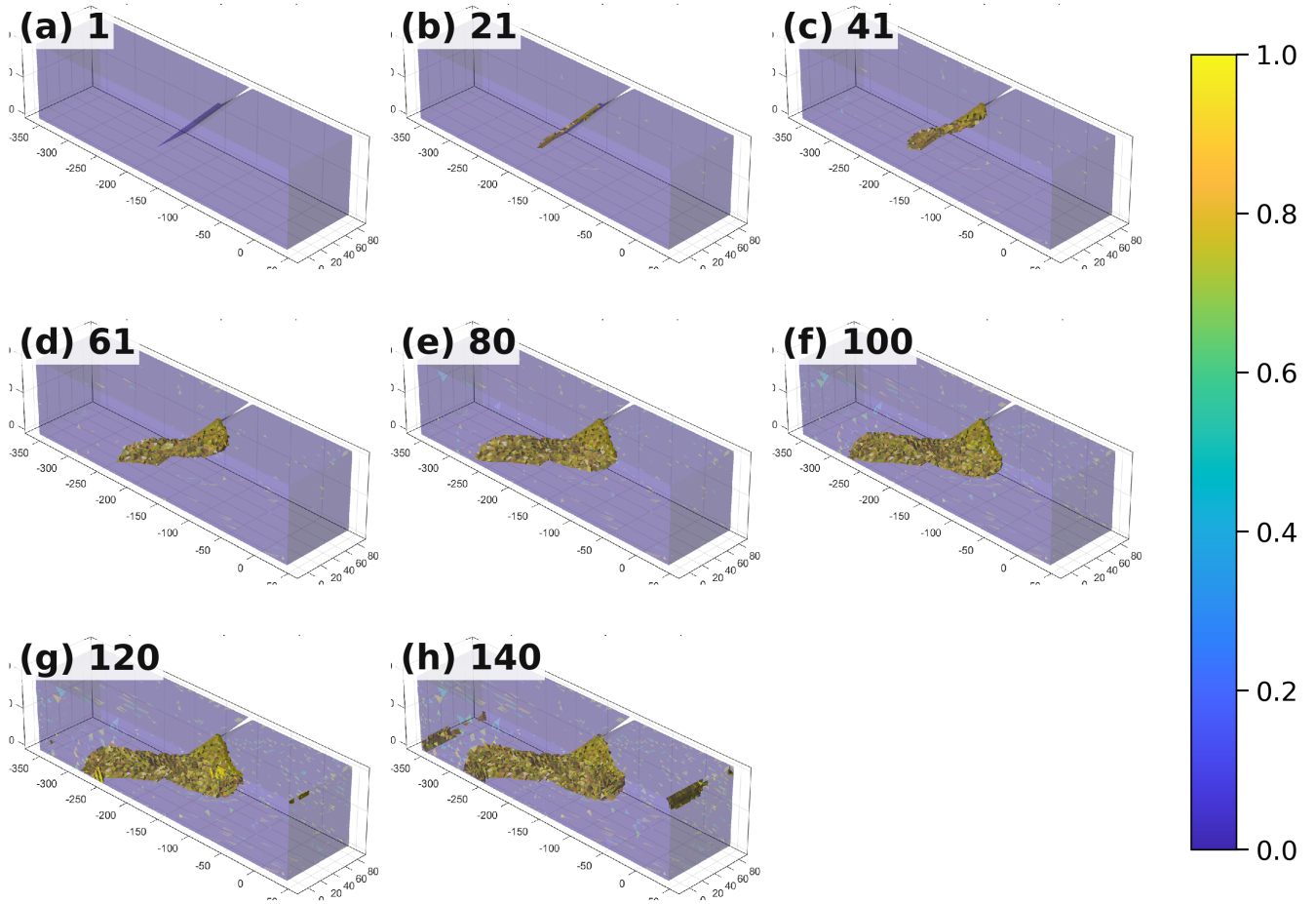
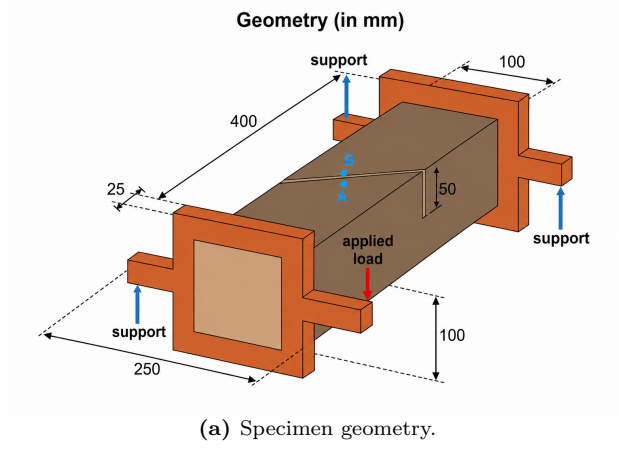


Figure 5: Brokenshire torsion example. The 3D panels illustrate the workflow qualitatively; quantitative validation against a measured torque curve is outside this study.

Controlled mesh sensitivity and computational scaling

A controlled 2D mesh family contains 14,313, 25,042, and 56,216 CPS3 elements, corresponding to 14,686, 25,534, and 56,954 DOFs. The notched-beam geometry and material properties are unchanged. Additional partitions place the supports exactly at $x = 50$ and 300 mm and define a fixed 6.25 mm loading strip at the top centre; all nodes on that strip receive the same displacement. The 50 mm-wide central zone containing the notch is refined across the beam height. Its nominal seeds are 1.25, 0.9375, and 0.625 mm; the outer seeds are five times larger. This concentrates elements near the expected damage path while retaining a coarser outer region. In all three main Oliver runs, elements with $\omega \geq 0.95$ at peak load have centroids inside this refined region, at heights 20.2–37.6 mm above the beam bottom. This checks the chosen refinement location, without asserting convergence of the damage-band geometry. These boundary sets and each complete Abaqus connectivity are exported directly to MATLAB; mesh-file hashes are checked after Abaqus rebuilds. This family is distinct from the single-mesh increment study. Both programs use one computational thread/CPU.

The main MATLAB runs impose -0.1 mm in 2,000 increments ($|\Delta u| = 5 \times 10^{-5}$ mm). The regularized runs update the Oliver projected width; the control runs hold the softening calibration width at $h_{\text{ref}} = 1.25$ mm on every mesh. This control uses one fixed stress–strain law and disables element-size compensation. A second complete MATLAB family halves $|\Delta u|$ on all three meshes for both calibrations, using 4,000 increments. The Abaqus jobs use the same maximum displacement and increment cap, with automatic cutbacks available; their accepted counts are reported.

For each MATLAB increment, the undamaged energy density is $Y_e = \frac{1}{2} \boldsymbol{\epsilon}_e^T \mathbf{D} \boldsymbol{\epsilon}_e$. Accumulated damage dissipation is evaluated by the positive end-of-step quadrature

$$\mathcal{D}_N = \sum_{n=1}^N \sum_e Y_e^n (\omega_e^n - \omega_e^{n-1}) V_e. \quad (4)$$

External work uses trapezoidal integration of reaction versus prescribed displacement; stored energy is $\sum_e (1 - \omega_e) Y_e V_e$. The discrepancy between external work and stored energy plus $\mathcal{D}_N$ is retained as a numerical diagnostic. Energy comparisons use a common CMOD of 0.10 mm and represent partial structural dissipation, not a measurement of complete fracture energy.

Figure 6 compares the two softening calibrations. Across the three main runs, peak-load spread, defined as $(\max - \min)/\text{mean}$, is 6.19% with Oliver regularization and 6.78% with the fixed law. The corresponding dissipation spreads are 6.86% and 18.21% (Table 5). The comparison tests element-size compensation for this mesh family; remaining sensitivity does not support complete mesh independence.

Table 5: Controlled mesh study, 2,000 MATLAB increments. Partial damage dissipation $\mathcal{D}$ is evaluated at CMOD 0.10 mm. Subscripts O, F, and A denote Oliver MATLAB, fixed-law MATLAB, and regularized Abaqus runs.

Elements	DOFs	P_O (kN)	P_F (kN)	P_A (kN)	$\mathcal{D}_O$ (N mm)	$\mathcal{D}_F$ (N mm)
14,313	14,686	4.194	4.025	4.117	140.08	132.13
25,042	25,534	4.205	3.808	4.087	140.93	117.25
56,216	56,954	4.459	3.763	4.339	149.94	110.30

At 4,000 increments on every mesh, peak-load spreads are 5.98% (Oliver) and 8.05% (fixed law); dissipation spreads are 6.44% and 20.76%, respectively. The reduction in dissipation sensitivity therefore persists at the smaller increment. Halving the displacement increment gives coarse fixed (-0.96% peak, -1.27% dissipation), coarse oliver (-0.89% peak, -0.99% dissipation), medium fixed (-1.85% peak, -2.55% dissipation), medium oliver (-1.37% peak, -1.53% dissipation), fine fixed (-2.20% peak, -3.74% dissipation), fine oliver (-1.31% peak, -1.42% dissipation). These finite-increment checks quantify sensitivity; they do not establish asymptotic convergence. At CMOD 0.10 mm, the absolute energy-balance discrepancy is 0.42–1.00% of external work in the main runs and 0.21–0.48% in the refined runs. The fine Oliver peak post-damage relative residual decreases from

0.0052 to 0.0029. The residual and energy discrepancy remain explicit limits of the sequential update.

Table 6: Single-CPU study timings in seconds. MATLAB has 2,000 fixed increments. Abaqus counts are accepted increments and solver passes. Factorization and sparse-solver timers have different scopes; these are single-run measurements, not an accuracy-matched speed ranking.

DOFs	MATLAB	Assembly	Factor.	Abaqus	Sparse	Incr.	Passes
14,686	150.9	30.8	102.8	890.0	95.8	2,008	5,223
25,534	233.5	45.2	157.5	1794.0	208.4	2,000	6,225
56,954	621.0	104.1	432.1	4585.0	605.3	2,000	7,566

MATLAB peak loads differ from the corresponding Abaqus peaks by +1.87%, +2.88%, +2.78%, respectively, relative to Abaqus. The completed Abaqus jobs report coarse: 2 cutbacks and 5,152 decompositions, medium: 0 cutbacks and 6,153 decompositions, fine: 0 cutbacks and 7,502 decompositions. Alternate force-tolerance acceptances are 4, 3, 5, respectively. Their wall time outside the sparse-solver timer is 794.2 s, 1585.6 s, 3979.7 s, respectively. It includes unseparated element/UMAT, assembly, convergence, output, and overhead; no material-time attribution is inferred. The fine-mesh response overlay is a code-to-code check with different equilibrium algorithms. Machine-readable histories, exact meshes, input decks, diagnostics, and analysis scripts accompany the study.

Impact

The main value of **FRACMATH** is that users can trace a complete damage calculation through mesh import, element matrices, projected bandwidth, constitutive history, sparse solve, and plotted response. The material-point test and the timing scopes are small, inspectable checks that expose two issues in the prior version: an incorrect 2D invariant coefficient and mislabelled factorization time. The 2D and 3D examples provide starting cases for method development and classroom use. The supplied scripts permit the figures to be regenerated from the associated data.

Limitations

The model is local, isotropic, quasistatic, and intended for monotonic loading. A single damage scalar does not represent crack closure or anisotropic degradation. The material-point tests verify softening calibration. The controlled 2D family tests mesh sensitivity for one geometry and mesh construction; it does not establish general mesh or orientation independence. The 3D examples remain qualitatively assessed. The 2D sequential update leaves a nonzero post-damage equilibrium residual; the archived increment study and new refinement checks quantify its effect. A fully coupled outer equilibrium iteration is future work. The current Abaqus UMAT gradient-table initialization is not safe for multithreaded analysis. The larger study reports completed-job diagnostics, but the algorithms and timing scopes differ. An accuracy-matched speed ranking and multithreaded scaling remain unverified; no GPU performance claim is made.

Conclusions

FRACMATH is an inspectable MATLAB example of local damage with an Oliver projected crack-band width. The study presents constitutive checks, controlled 2D mesh and increment studies, structural energy diagnostics, and timing scopes that separate sparse factorization from assembly. The manuscript reports the solver’s present limitations and retains the 3D examples as qualitative demonstrations.

Data availability

The 2D MATLAB source, UMAT source, mesh, material-point test, machine-readable response histories, and figure-generation script are available in the [FRACMATH GitHub repository](#). The code and study data are identified by the immutable Git commit listed in Table 1.

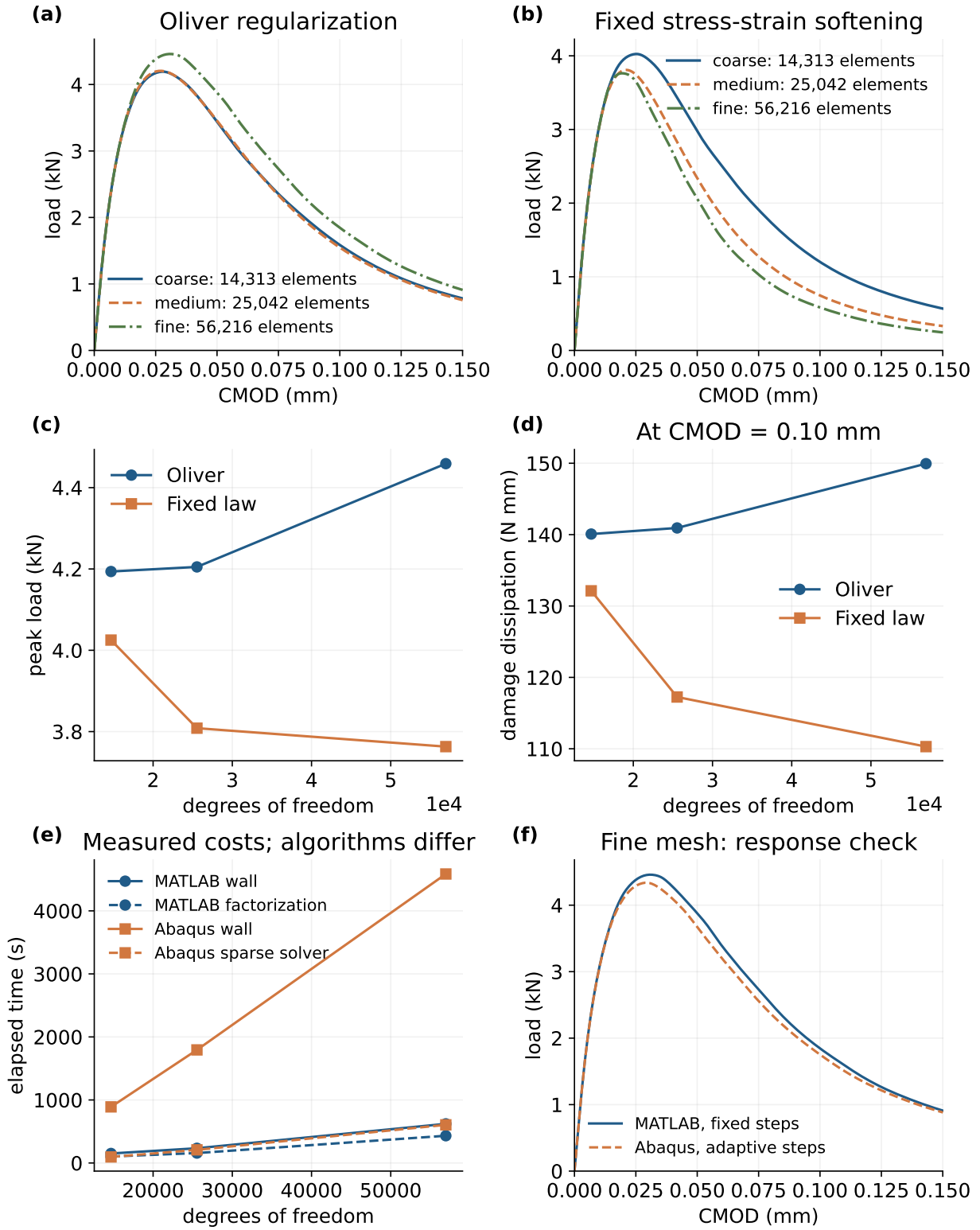


Figure 6: Controlled 2D study. (a,b) Oliver and fixed-law responses; (c,d) peak load and partial damage dissipation versus DOFs; (e) measured wall and linear-algebra costs; (f) fine-mesh MATLAB/Abaqus responses. Main MATLAB curves use 2,000 increments to -0.1 mm; Abaqus uses the same maximum increment with adaptive cutbacks. Cost scopes and equilibrium algorithms differ.

CRedit authorship contribution statement

Jaykumar Mavani: Methodology, Software, Validation, Visualization, Writing – original draft.
Madura Pathirage: Conceptualization, Supervision, Funding acquisition.

Acknowledgements

The authors acknowledge support from the National Aeronautics and Space Administration (NASA), and the New Mexico Space Grant Consortium under grant 80NSSC22M0044.

Declaration of competing interest

The authors declare no conflict of interest.

Declaration of AI-assisted technologies

Generative AI tools were used to assist with manuscript wording, formatting, and implementation support. The scientific claims, equations, code, numerical results, figures, and references are the responsibility of the authors.

References

- [1] J. H. P. de Vree, W. A. M. Brekelmans, M. A. J. van Gils, Comparison of nonlocal approaches in continuum damage mechanics, *Computers & Structures* 55 (4) (1995) 581–588. doi:10.1016/0045-7949(94)00501-S.
- [2] J. Oliver, A consistent characteristic length for smeared cracking models, *International Journal for Numerical Methods in Engineering* 28 (2) (1989) 461–474. doi:10.1002/nme.1620280214.
- [3] Z. P. Bažant, B. H. Oh, Crack band theory for fracture of concrete, *Matériaux et Construction* 16 (3) (1983) 155–177. doi:10.1007/BF02486267.
- [4] Dassault Systèmes Simulia Corp., *Abaqus/Standard User’s Manual*, Version 2024, Providence, RI, USA, 2024.
- [5] B. Patzák, Z. Bittnar, Design of object-oriented finite element code, *Advances in Engineering Software* 32 (10–11) (2001) 759–767. doi:10.1016/S0965-9978(01)00027-8.
- [6] P.-E. Austrell, O. Dahlblom, J. Lindemann, A. Olsson, K.-G. Olsson, K. Persson, H. Petersson, M. Ristinmaa, G. Sandberg, P.-A. Wernberg, *CALFEM—A Finite Element Toolbox*, Version 3.4, Lund University, Lund, Sweden, 2004.
- [7] D. Grégoire, L. B. Rojas-Solano, G. Pijaudier-Cabot, Failure and size effect for notched and unnotched concrete beams, *International Journal for Numerical and Analytical Methods in Geomechanics* 37 (10) (2013) 1434–1452. doi:10.1002/nag.2180.
- [8] M. B. Nooru-Mohamed, Mixed-mode fracture of concrete: an experimental approach, Ph.D. thesis, Delft University of Technology, 1992. URL: <https://resolver.tudelft.nl/uuid:a6a773f1-dacd-4598-aa6a-960dddf71117>.
- [9] D. R. Brokenshire, A study of torsion fracture tests, Ph.D. thesis, Cardiff University, Cardiff, United Kingdom, 1996.
- [10] A. D. Jefferson, B. I. G. Barr, T. Bennett, S. C. Hee, Three dimensional finite element simulations of fracture tests using the Craft concrete model, *Computers and Concrete* 1 (3) (2004) 261–284. doi:10.12989/cac.2004.1.3.261.